DAY 16 | Deployments & Rollouts

90 Days Challenge | Week 3: Kubernetes | Release • Observe • Break • Roll Back

LEVEL	TIME	FOCUS
Beginner / Advanced beginner	3–4 hours	Rolling updates, rollout history, release annotations, failed rollouts and rollback verification

GOAL OF THE DAY

Practice safe application releases on your existing local Kubernetes cluster. Update a workload, inspect rollout progress and revision history, simulate a failed image update, and recover using a rollback.

YOUR TASKS TODAY

- Use the cluster and workload from your previous Kubernetes lab if they are still available; do not recreate resources unnecessarily.
- Record a release change-cause before updating the image.
- Perform a rolling update and verify its rollout status.
- Inspect revision history and compare saved revisions.
- Trigger a deliberately invalid image update, inspect the failure, then roll back.
- Explain maxSurge and maxUnavailable using your observed rollout.

DELIVERABLE

A GitHub folder named day16-rollouts/ containing a README, command output or screenshots for the successful update, failed rollout, revision history and rollback verification.

SCOPE RULE — READ BEFORE STARTING

Day 15 already covered Kubernetes fundamentals, local cluster setup, basic Deployment and Service YAML, Nginx deployment, scaling replicas, and basic access testing. These topics are intentionally not repeated as tasks here. This assignment focuses on the remaining Day 1–Day 2 material from the Week 3 PPT, especially release history, rolling-update controls, failure observation and rollback.

Safety note

Use a local learning cluster only. Do not run these commands against production. The intentionally broken image is for a temporary lab workload; roll it back after collecting evidence.

01 / UNDERSTAND ROLLOUTS

Concepts you need for this lab

CONCEPT	WHAT IT MEANS	WHAT YOU WILL VERIFY
Rolling update	Gradually replaces old Pods with Pods from a new Pod template.	The new revision becomes available.
ReplicaSet revisions	A Deployment creates/updates ReplicaSets to represent versions of its Pod template.	History contains release changes.
maxSurge	Maximum extra Pods allowed above the desired replica count during an update.	Temporary extra capacity may be used.
maxUnavailable	Maximum desired Pods allowed to be unavailable during an update.	Availability is controlled during rollout.
Change-cause annotation	A note describing why a release was made.	The release reason is visible in history where supported.
Rollback	Return the Deployment to a previous saved revision.	The working image/version is restored.

Commands to know

COMMAND	PURPOSE
kubectl rollout status deployment/nginx-deployment	Wait for rollout and report whether it completed.
kubectl rollout history deployment/nginx-deployment	List saved revisions.
kubectl rollout history deployment/nginx-deployment --revision=2	Inspect a revision; replace 2 with an available revision.
kubectl annotate deployment/nginx-deployment kubernetes.io/change-cause="..."	Record why a release is being made.
kubectl set image deployment/nginx-deployment nginx=nginx:1.28	Change the image and start a rollout; use your actual container name.
kubectl rollout undo deployment/nginx-deployment	Return to the previous revision.
kubectl rollout undo deployment/nginx-deployment --to-revision=2	Return to a chosen saved revision.
kubectl describe pod POD_NAME	Inspect Events and diagnose a failed Pod.
kubectl get rs,pods -o wide	Observe ReplicaSets and Pods during release/recovery.

Use the Deployment and container names that exist in your cluster. If your Day 15 resources were deleted, create only a small lab workload as needed; do not repeat the full Day 15 assignment.

02 / RUN A CONTROLLED RELEASE

Step 1 — Find your existing Deployment

```
kubectl config current-context
kubectl get deployments
kubectl get pods
kubectl get rs
```

The commands below use nginx-deployment and container nginx as examples. Replace them if your resources have different names.

Step 2 — Inspect the rollout strategy

```
kubectl describe deployment nginx-deployment
kubectl get deployment nginx-deployment -o yaml
```

Find the strategy settings. If maxSurge or maxUnavailable are not specified, Kubernetes uses the defaults for RollingUpdate.

Step 3 — Annotate and update the image

```
kubectl annotate deployment/nginx-deployment \
  kubernetes.io/change-cause="Day 16: update Nginx for rollout practice"

kubectl set image deployment/nginx-deployment \
  nginx=nginx:1.28

kubectl rollout status deployment/nginx-deployment
kubectl rollout history deployment/nginx-deployment
kubectl get rs,pods -o wide
```

Step 4 — Verify the release

- Rollout status reports success.
- The Deployment's current image shows nginx:1.28 or the valid version you chose.
- Rollout history shows a new revision; inspect its details.
- Pods settle into their expected Running/Ready state.

Evidence to save

```
kubectl rollout status deployment/nginx-deployment
kubectl rollout history deployment/nginx-deployment
kubectl get deployment nginx-deployment -o wide
kubectl get rs,pods -o wide
```

> If the image tag is unavailable, choose a valid tag and record which one you used. The goal is to observe rollout and revision tracking.

03 / BREAK A RELEASE & ROLL BACK

Step 5 — Save the known-good revision

```
kubectl rollout history deployment/nginx-deployment
kubectl get deployment nginx-deployment \
  -o jsonpath='{.spec.template.spec.containers[0].image}'; echo
```

Write down the working image and current revision in your README.

Step 6 — Trigger a temporary failed rollout

```
kubectl set image deployment/nginx-deployment \
  nginx=nginx:1.99-broken

kubectl get pods -w
```

Wait briefly for new Pods to show a pull failure, then press Ctrl+C. Old healthy Pods may remain available while the new ReplicaSet fails to become ready.

Step 7 — Diagnose the failure

```
kubectl get pods
kubectl describe pod POD_NAME
kubectl get events --sort-by=.metadata.creationTimestamp
kubectl rollout history deployment/nginx-deployment
```

Replace POD_NAME with a Pod from the failed rollout. Find the image-pull error in Events and explain why the new version could not start.

Step 8 — Roll back

```
kubectl rollout undo deployment/nginx-deployment
kubectl rollout status deployment/nginx-deployment
kubectl rollout history deployment/nginx-deployment
kubectl get deployment nginx-deployment -o wide
kubectl get pods
```

Verify the Deployment has returned to the last known-good image and its Pods become ready. If multiple revisions exist, use --to-revision=REVISION to select a specific revision.

> Do not leave the broken image deployed. If rollout hangs, stop watching with Ctrl+C, run rollout undo, and verify the restored version.

04 / TROUBLESHOOTING & SUBMISSION

PROBLEM	CHECK / FIX
Rollout status never completes	Inspect Pods and run kubectl describe pod POD_NAME; read Events for image or readiness errors.
ImagePullBackOff	Confirm the image name/tag. Here, the invalid tag is intentional; roll back after collecting evidence.
History does not show the expected note	Check that change-cause annotation was applied before the image update. Inspect history again.
Wrong Deployment/container name	Use kubectl get deployments and kubectl describe deployment DEPLOYMENT_NAME.
Rollback seems not to work	Inspect current image and Pods, wait for rollout status, and check the selected revision.
No previous revision exists	Make a successful Pod-template change first; history records revisions after template changes.

Submission checklist

- ■ README explains rolling updates, revision history, maxSurge, maxUnavailable and rollback in your own words.
- ■ Evidence includes successful rollout status and revision history.
- ■ Evidence includes the failed image update and Pod Events that explain it.
- ■ Evidence proves rollback restored the known-good image and healthy Pods.
- ■ No kubeconfig, tokens, cloud credentials or other secrets are committed.

Suggested repository structure

```
day16-rollouts/
  README.md
  evidence/
    rollout-success.png
    rollout-history.png
    failed-rollout-events.png
    rollback-verified.png
```

Cleanup

If you reused the Day 15 Deployment, restore it to the image/version you want to keep. Delete resources only if created solely for this lab.

```
# Optional: delete only if this lab created the Deployment
kubectl delete deployment nginx-deployment
```

05 / INTERVIEW READINESS

QUESTION	KEY IDEA
What does a rolling update do?	Gradually replaces old Pods with the new Pod template while applying availability limits.
What are maxSurge and maxUnavailable?	Controls extra Pods allowed and desired Pods allowed to be unavailable during an update.
Why inspect rollout history?	It shows revisions that can be inspected and used for rollback.
How do you diagnose a failed rollout?	Inspect Pods, describe a failing Pod and read Events/logs.
How does rollout undo work?	It restores a previous saved Deployment revision.
Why annotate a release?	It records the reason for a change and makes release history easier to understand.

BONUS CHALLENGE

Configure a deliberate RollingUpdate strategy with maxSurge: 1 and maxUnavailable: 0 in a separate lab manifest. Apply it, perform one valid update, then explain how the settings affect availability. Do not overwrite shared or production resources.

REVIEW CRITERIA (100 POINTS)

AREA	POINTS
Successful update and rollout status evidence	20
Revision history and release annotation	20
Failed rollout diagnosed with Events	20
Rollback verified to a known-good image	25
README, screenshots and safe cleanup	15

FINAL DELIVERABLE: Submit your GitHub repository link and evidence screenshots. Record only results from your own cluster; do not fabricate successful outputs.